A1-10 shared page – the short variant

Not tested yet. This is a shorter version of the tested solution (`solution.patch`, explained in `walkthrough.pdf`): no new class, no new files, one extra lock. Run `task test` – done when **9 of 9** pass. If it does not, compare with `minimal.pdf` (tested).

The idea

- Two processes that attach the same key get the **same physical page** in their page tables.
- Two global arrays remember, per key, **which physical page** and **how many processes** use it.
- Free the page only when the count reaches **0**; unmapping a shared page never frees it.

What is shorter than the tested solution

	tested solution	short variant
the table	class `SharedPages`, 2 new files	2 arrays + 1 lock pointer in `UserProcess.cpp`
locks	two locks, never both at once	both at once, **always in the same order**
lines of logic	≈ 90	≈ 45

The one rule that keeps it deadlock-free: wherever both locks are needed, take `shm_lock` **first**, then `thread_lock_`. Nothing else in the kernel takes `shm_lock`, so no thread can ever hold `thread_lock_` and wait for `shm_lock`.

1 – Plumbing `syscall-definitions.h · nonstd.h · nonstd.c · Syscall.cpp`

```
// syscall-definitions.h
#define sc_shm_attach 1680
#define sc_shm_detach 1681

// nonstd.h  (needs #include "types.h" at the top)
extern void* shm_attach(size_t key);
extern int shm_detach(size_t key);

// nonstd.c
void* shm_attach(size_t key)
{
  return (void*) __syscall(sc_shm_attach, key, 0x00, 0x00, 0x00, 0x00);
}
int shm_detach(size_t key)
{
  return __syscall(sc_shm_detach, key, 0x00, 0x00, 0x00, 0x00);
}

// Syscall.cpp, in the switch of syscallException
case sc_shm_attach:
  return_value = currentThread->getProcess()->shmAttach(arg1);
  break;
case sc_shm_detach:
  return_value = currentThread->getProcess()->shmDetach(arg1);
  break;
```

2 – Unmap without freeing

ArchMemory.h · ArchMemory.cpp

```
// ArchMemory.h
bool unmapPage(uint64 virtual_page, bool free_physical_page = true);

// ArchMemory.cpp
bool ArchMemory::unmapPage(uint64 virtual_page, bool free_physical_page)
  ...
  if (free_physical_page)
    PageManager::instance()->freePPN(m.page_ppn);
```

Why: the physical page still belongs to the other processes.

3 – Constants and per-process state

UserProcess.h

```
// at the top, next to the other #defines
#define SHM_KEYS 16                          // keys 1..15 (0 is unused)
#define SHM_BASE 0x0000400000000000ULL       // key k lives at SHM_BASE + k pages

// public:
size_t shmAttach(size_t key);
size_t shmDetach(size_t key);

// private, at the bottom:
bool shm_attached_[SHM_KEYS] = {};           // which keys THIS process has mapped
```

4 – The table: two arrays and a lock

UserProcess.cpp (top) · main.cpp

```
// UserProcess.cpp, below the #includes (add #include "ScopeLock.h" if it is missing)
size_t shm_ppn[SHM_KEYS] = {};      // key -> physical page, 0 = none yet
size_t shm_refs[SHM_KEYS] = {};     // key -> how many processes have it attached
Mutex* shm_lock = nullptr;          // protects both arrays; created in startup()
```

- Plain arrays of numbers need no constructor – they are simply zero at boot.
- A `Mutex` *does* need its constructor, and SWEB never runs constructors of global objects – so only a pointer here, the object is created at boot:

```
// main.cpp: #include "Mutex.h" at the top, and in startup() after
    ↪ Scheduler::instance();
extern Mutex* shm_lock;
shm_lock = new Mutex("shm_lock");
```

(`extern Mutex* shm_lock;` goes *above* `startup()`, at file level: "this variable exists in another file".)

5 – Attach

UserProcess.cpp

```
size_t UserProcess::shmAttach(size_t key)
{
  if (key == 0 || key >= SHM_KEYS)
    return 0;
  size_t vpn = SHM_BASE / PAGE_SIZE + key;
  ScopeLock global(*shm_lock);          // 1st lock
  ScopeLock local(thread_lock_);        // 2nd lock - always in this order
  if (!shm_attached_[key])
  {
    if (shm_refs[key] == 0)
      shm_ppn[key] = PageManager::instance()->allocPPN();   // first user: new zeroed
    ↪ page
```

```
    shm_refs[key]++;
    bool mapped = arch_memory_->mapPage(vpn, shm_ppn[key], 1);
    assert(mapped);
    shm_attached_[key] = true;
  }
  return vpn * PAGE_SIZE;               // attached twice: same address, counted once
}
```

6 – Detach

UserProcess.cpp

```
size_t UserProcess::shmDetach(size_t key)
{
  if (key == 0 || key >= SHM_KEYS)
    return (size_t) -1;
  ScopeLock global(*shm_lock);
  ScopeLock local(thread_lock_);
  if (!shm_attached_[key])
    return (size_t) -1;
  arch_memory_->unmapPage(SHM_BASE / PAGE_SIZE + key, false);   // keep the physical
    ↪ page
  ArchMemory::flushTlb();
  shm_attached_[key] = false;
  shm_refs[key]--;
  if (shm_refs[key] == 0)
    PageManager::instance()->freePPN(shm_ppn[key]);             // last user: free it
  return 0;
}
```

7 – Clean up when a process ends

UserProcess.cpp, ~UserProcess

Between `delete loader_;` and `delete arch_memory_;`:

```
{
  ScopeLock global(*shm_lock);
  for (size_t key = 1; key < SHM_KEYS; ++key)
  {
    if (!shm_attached_[key])
      continue;
    arch_memory_->unmapPage(SHM_BASE / PAGE_SIZE + key, false);
    shm_refs[key]--;
    if (shm_refs[key] == 0)
      PageManager::instance()->freePPN(shm_ppn[key]);
  }
}
```

- **Why:** `delete arch_memory_` frees every mapped page – also a shared page the other process still uses, if this one ends without detaching.
- The extra `{ }` ends the `ScopeLock` before `delete arch_memory_`.
- No `thread_lock_` here: the process is being deleted, none of its threads runs any more.

Remember

1. Count the users per key; free the page only at 0.
2. Unmapping a shared page never frees it (`false`).
3. Two locks: always `shm_lock` first, then `thread_lock_` – the same order everywhere.